



Data Engineering

MINI PROJECT



Sales Data Warehousing & Transformation

Mini Project: Build a Modern Data Warehouse with Snowflake, dbt, and Git

Project Title

Retail Sales Data Warehouse using Snowflake + dbt + GitHub

Project Overview

In this mini project, you will act as a **Data Engineer** working for a retail company that wants to modernize its data warehouse.

The company currently receives daily sales data as CSV files. Your responsibility is to build an analytics-ready data warehouse using **Snowflake** and **dbt**, while managing the project with **Git** and **GitHub**.

By the end of this project, you will transform raw sales data into clean, well-structured business models that can be used by analysts and dashboard developers.

Business Scenario

ABC Retail is a growing e-commerce company selling electronics, clothing, home appliances, books, and sports equipment.

Every day, the company exports four CSV files:

 Customers

 Products

 Orders

 Order Items

Management wants answers to questions such as:

 Which products generate the highest revenue?

- ☀️ Who are the top customers?
- 📊 Which product categories perform best?
- 📅 How much revenue is generated each month?
- 🌍 Which countries have the most sales?

Unfortunately, the raw CSV files are inconsistent, contain duplicates, and are difficult to analyze.

Your task is to build a modern analytics pipeline.

🎯 Project Objectives

You will:

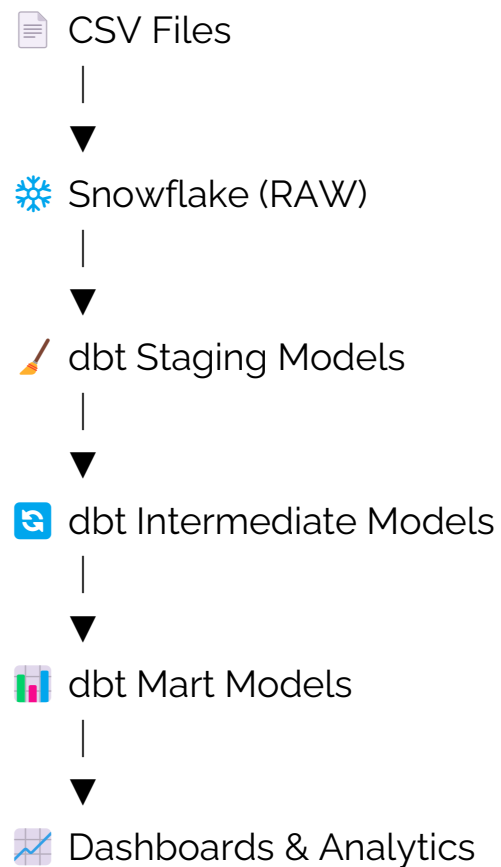
- ✅ Load raw CSV files into Snowflake
- ✅ Organize data into schemas
- ✅ Build staging models using dbt
- ✅ Build intermediate models
- ✅ Build mart models
- ✅ Test data quality
- ✅ Document the project
- ✅ Track changes using Git
- ✅ Publish the project to GitHub

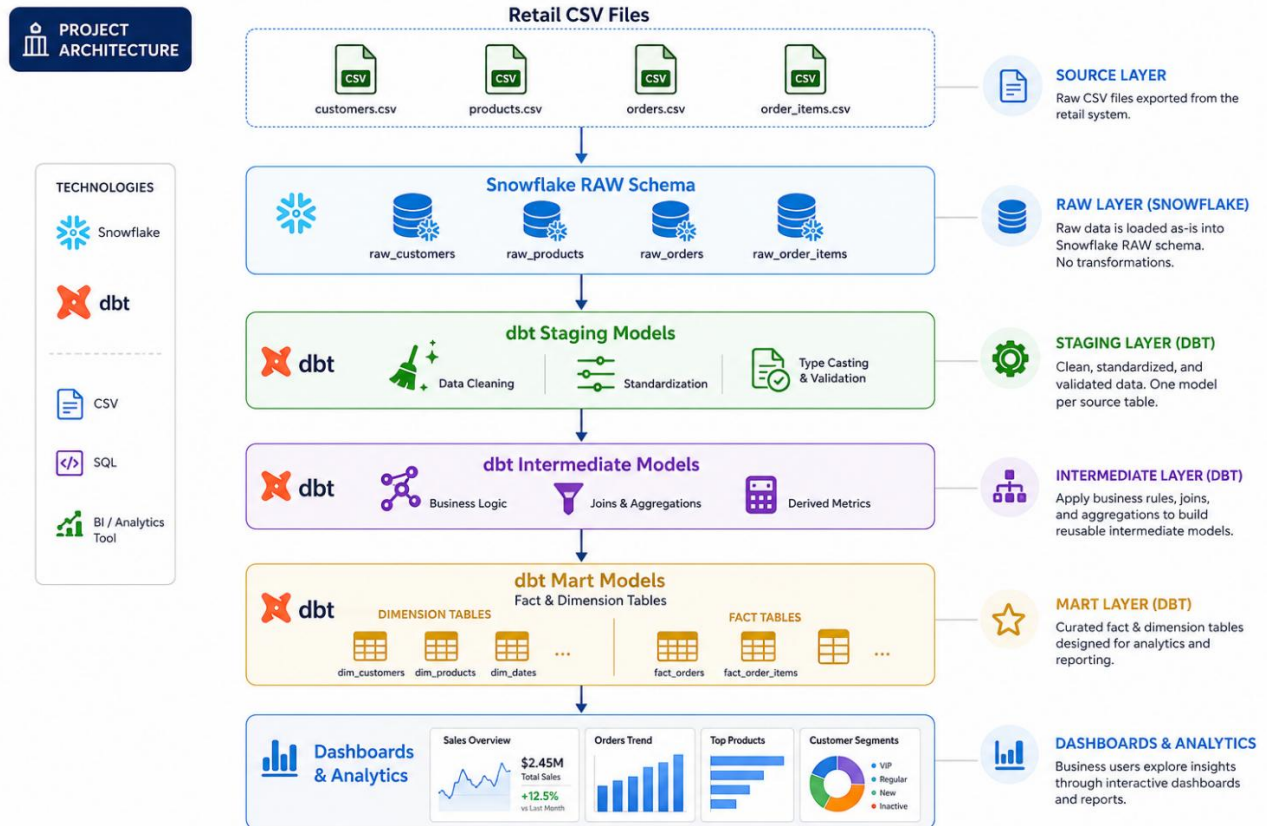
⚙️ Technologies Used

TECHNOLOGY	PURPOSE
 SNOWFLAKE	Cloud Data Warehouse

 DBT CORE	Data Transformation
 GIT	Version Control
 GITHUB	Code Repository
 VS CODE	Development
 SQL	Query Language
 CSV FILES	Data Source

Project Architecture





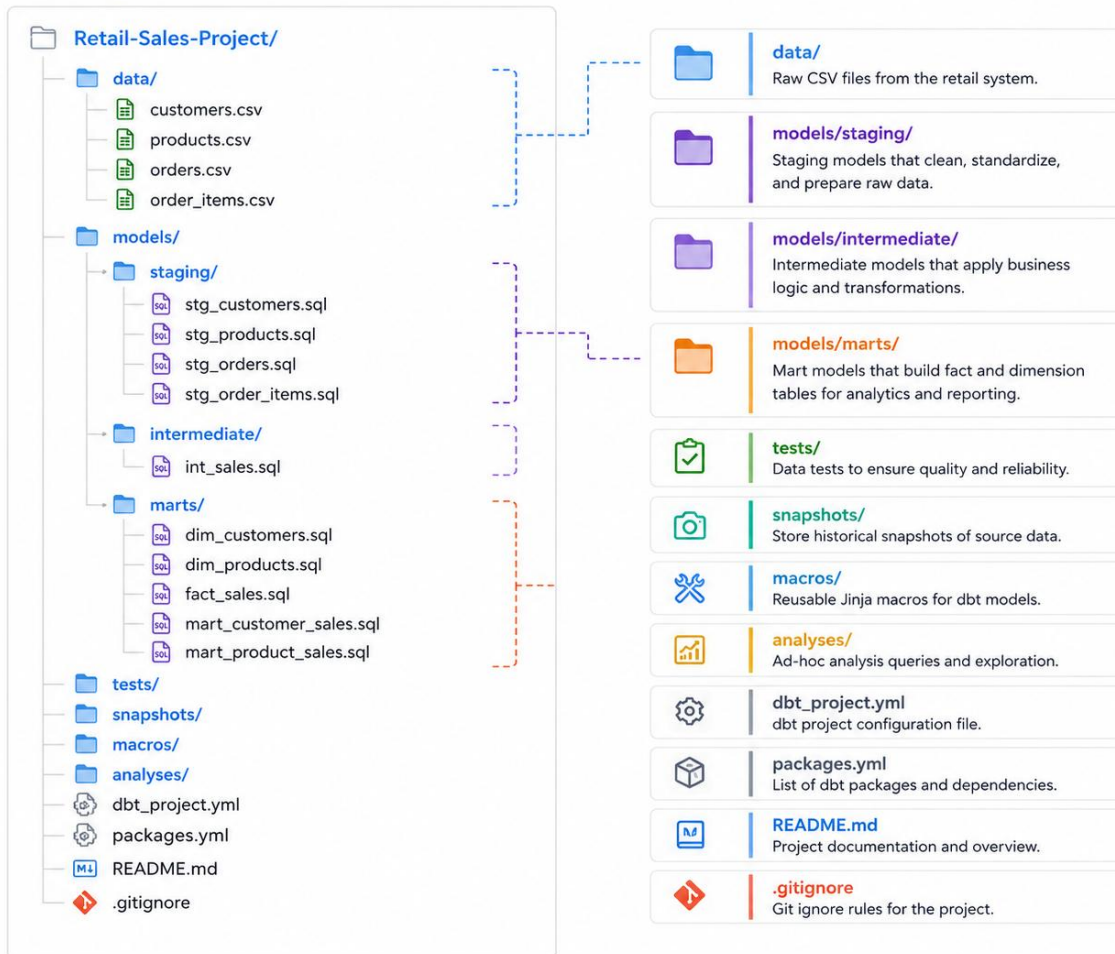
Project Folder Structure

Retail-Sales-Project/

- data/
- models/
 - staging/
 - intermediate/
 - marts/
- tests/
- snapshots/
- macros/
- analyses/

- dbt_project.yml
- packages.yml
- README.md
- .gitignore

Project Folder Structure



Dataset

Students receive four CSV files.

customers.csv

Column	Description
customer_id	Unique Customer ID
first_name	First Name
last_name	Last Name
email	Email Address
city	City
country	Country

products.csv

Column	Description
product_id	Product ID
product_name	Product Name
category	Product Category
price	Product Price

orders.csv

Column	Description
order_id	Order ID
customer_id	Customer
order_date	Purchase Date

order_items.csv

Column	Description
--------	-------------

order_item_id	Item ID
---------------	---------

order_id	Order
----------	-------

product_id	Product
------------	---------

quantity	Quantity
----------	----------

Part 1 — Git & GitHub Setup

Tasks

 Create project folder

Retail-Sales-Project

 Initialize Git

git init

 Create .gitignore

target/

logs/

dbt_packages/

.env

 Create GitHub repository

retail-sales-dbt

 Connect local repository to GitHub

git remote add origin <https://github.com/username/retail-sales-dbt.git>

 Make the initial commit


```
git add .
```

```
git commit -m "Initial project structure"
```

 Push code to GitHub

```
git push -u origin main
```

Part 2 — Snowflake Setup

Tasks

Create:

 Database

RETAIL_DB

 Warehouse

COMPUTE_WH

 RAW Schema

 STAGING Schema

 INTERMEDIATE Schema

 MART Schema

Then:

 Upload

Load the CSV files into the **RAW** schema.

- customers.csv
- products.csv

- orders.csv
- order_items.csv

Required tables

customers

products

orders

order_items

Verify the data

```
SELECT * FROM RAW.CUSTOMERS;
```

```
SELECT * FROM RAW.PRODUCTS;
```

```
SELECT * FROM RAW.ORDERS;
```

```
SELECT * FROM RAW.ORDER_ITEMS;
```

Part 3 — Configure dbt

Tasks

 Initialize dbt project

```
dbt init retail_project
```

 Configure Snowflake connection

```
profiles.yml
```

✓ Test connection

dbt debug

▶ Run dbt project

dbt run

Create Sources

Create

sources.yml

Define

- customers
- products
- orders
- order_items

These become dbt source tables.

🔨 Part 4 — Build Staging Models

Create

👤 stg_customers.sql

📦 stg_products.sql

🛒 stg_orders.sql

📋 stg_order_items.sql

Each model should:

✓ Rename columns

✓ Remove duplicates

- ✓ Trim whitespace
- ✓ Handle NULL values
- ✓ Convert data types
- ✓ Standardize naming conventions

Example transformations:

first_name

↓

customer_first_name

PRICE

↓

product_price

Customer Name

↓

UPPER()

Email

↓

LOWER()

Run

dbt run

Part 5 — Build Intermediate Models

Create

 int_sales.sql

Tasks

 Join all staging models

 Calculate Revenue

 Build Customer Name

 Include Product Information

 Include Order Date

 Include Country

Join

- Customers
- Orders
- Products
- Order Items

Calculate

Revenue

Quantity

Unit Price

Customer Name

Product Name

Category

Country

Formula

Revenue = Quantity × Price

Output should resemble

Order Customer Product Quantity Revenue

Part 6 — Build Mart Models

Create analytical models.

Create

 **fact_sales**

Contains:

- order_id
- customer_id
- product_id
- quantity
- revenue
- order_date

 **dim_customers**

Contains:

- Customer Name
 - Email
 - City
 - Country
-

dim_products

Contains:

- Product Name
 - Category
 - Price
-

mart_customer_sales

Metrics

-  Total Revenue
 -  Products Purchased
 -  Total Orders
 -  Average Order Value
-

mart_product_sales

Metrics

-  Revenue
-  Units Sold

 Orders

 Best Selling Products

✓ Part 7 — Data Quality Testing

Create

 schema.yml

Run tests

✓ Unique

customer_id

product_id

order_id

⊘ Not Null

customer_name

price

quantity

 Relationships

Orders

↓

Customer

Order Items

↓

Orders

Order Items



Products

Execute

dbt test

Expected

PASS = 100%

Part 8 — Documentation

Generate documentation

dbt docs generate

Serve documentation

dbt docs serve

Verify

 Lineage Graph

 Model Descriptions

 Column Descriptions

 Test Results & Sources

Part 9 — Git Workflow

Commit after each milestone.

Example commits

 Initial Project Setup

 Create Snowflake Database

 Load Raw Data

 Configure dbt

 Create Staging Models

 Create Intermediate Models

 Create Mart Models

 Add Data Quality Tests

 Generate Documentation

 Final Project Submission

Push changes

git push

```
git add .
git commit -m "Create Snowflake database and schemas"

git add .
git commit -m "Load raw CSV data into Snowflake"

git add .
git commit -m "Configure dbt project"

git add .
git commit -m "Create staging models"

git add .
git commit -m "Create intermediate sales model"
```

```
git add .
git commit -m "Create mart models"

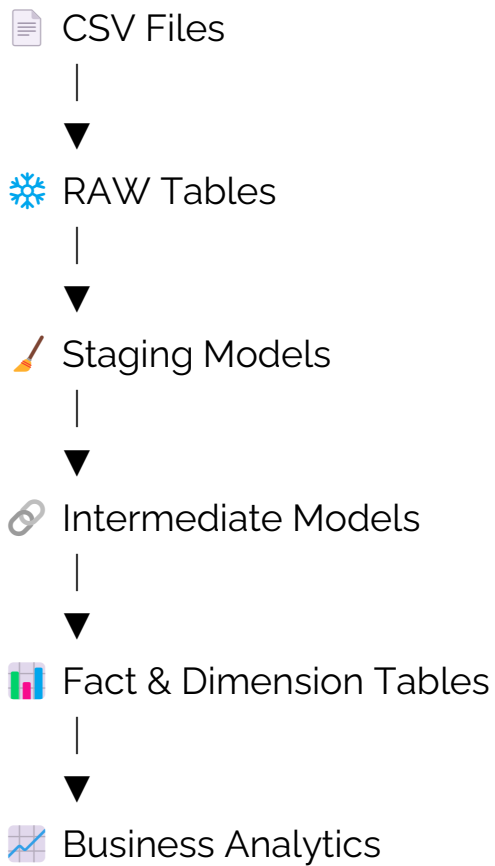
git add .
git commit -m "Add dbt tests"

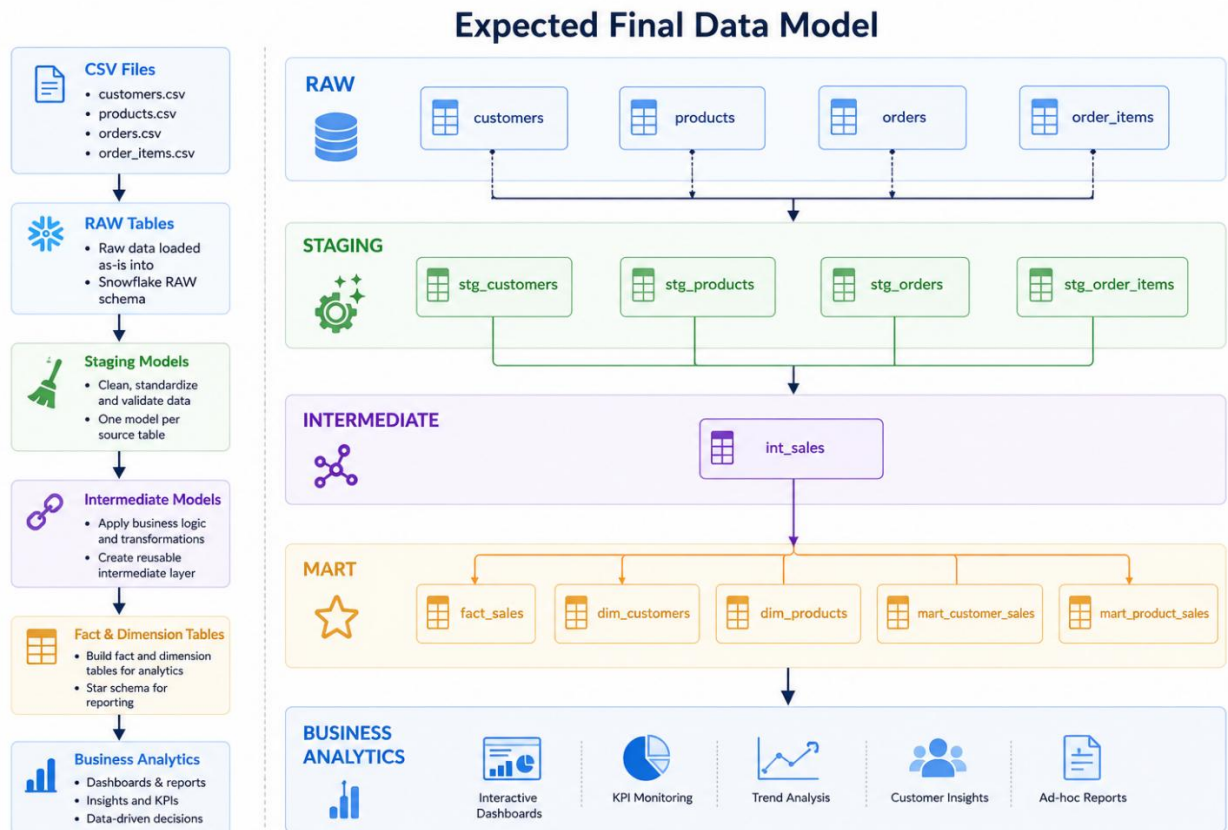
git add .
git commit -m "Generate documentation"

git push
```

git push

🔗 Expected Final Data Pipeline





Deliverables

Students must submit:

1. GitHub Repository

- Complete source code
- Meaningful commit history
- Organized folder structure

2. Snowflake Objects

- Database
- Warehouse

- Schemas
- Raw tables

3. dbt Project

- All models
- Source definitions
- Tests
- Documentation

4. Documentation

- README.md containing:
 - Project overview
 - Architecture diagram
 - Technology stack
 - Setup instructions
 - Screenshots
 - Data model
 - Challenges encountered
 - Lessons learned